Topic 2: Arrays and Dynamic Memory

2.1 Static Arrays

- Declared with a fixed size known at compile time: int arr[10];
- Stored in contiguous memory -- index access is O(1): arr[i] = base_address + i * sizeof(int)
- Size cannot change after declaration -- wasted space if too large, overflow if too small
- Pass to functions as a pointer: void func(int* arr, int size)

```
#include <iostream>
using namespace std;

void printArray(int* arr, int n) {
    for (int i = 0; i < n; i++) cout << arr[i] << " ";
    cout << endl;
}

int main() {
    int a[5] = {10, 20, 30, 40, 50};
    printArray(a, 5);
}
```

2.2 Dynamic Arrays with new / delete

- Allocate on the heap when size is not known at compile time
- int* arr = new int[n]; -- allocates n integers on the heap
- Always delete[] arr; when done to avoid memory leaks
- Resize requires: allocate larger array, copy old data, delete old array

```
int n;  cin >> n;
int* arr = new int[n];
for (int i = 0; i < n; i++) arr[i] = i * 2;

// Resize to 2n
int* bigger = new int[2*n];
for (int i = 0; i < n; i++) bigger[i] = arr[i];
delete[] arr;
arr = bigger;
// ... later ...
delete[] arr;
```

2.3 2D Arrays and Dynamic 2D Arrays

- Static 2D: int matrix[3][4]; -- stored row-major in contiguous memory
- Dynamic 2D: array of pointers, each pointing to a dynamically allocated row
- Useful when rows have different lengths (jagged arrays)

```
int rows = 3, cols = 4;
int** mat = new int*[rows];
for (int i = 0; i < rows; i++) mat[i] = new int[cols];

mat[1][2] = 99;

for (int i = 0; i < rows; i++) delete[] mat[i];
delete[] mat;
```

2.4 std::vector as a Dynamic Array

- vector<int> grows automatically -- no manual memory management
- push_back: O(1) amortised -- occasionally doubles capacity
- size() vs capacity(): size is elements stored; capacity is allocated space
- reserve(n): pre-allocate n slots to avoid repeated reallocation

---